

# 세르파 앱 프로덕션 청사진: **LangGraph** 오케스트레이션 기반 멀티 에이전트 **AI** 코치의 풀스택 전환

## I. 아키텍처 진화의 당위성: 모놀리식 **API**에서 에이전트 협업으로

### A. 'As-Is' 아키텍처 분석 (현상 유지)

현재 세르파 앱의 AI 분석 시스템은 명확한 4계층 아키텍처(UI Presentation, Service, Infrastructure, Models)를 기반으로 구축되어 있습니다.<sup>1</sup> 이 시스템의 핵심 기능은 **AiAnalysisScreen**에서 사용자의 분석 요청을 받아, 9단계의 데이터 흐름을 거쳐 처리하는 것입니다.<sup>1</sup>

현재 데이터 흐름은 **AiAnalysisDataCollector**를 통해 사용자의 신체 정보, 목표, 루틴 및 최근 2개월간의 러닝 기록을 수집합니다.<sup>1</sup> 이후 **AIPromptBuilder**는 이 모든 데이터를 "15년 경력의 프로 러닝 코치" 역할을 수행하도록 설계된 정교한 10개 섹션의 프롬프트로 조합합니다.<sup>1</sup>

이 아키텍처의 결정적인 단계는 8단계인 **OpenAIService**의 **createChatCompletion** 메서드 호출입니다.<sup>1</sup> 이 단계에서 시스템은 준비된 모든 컨텍스트를 **OpenAI GPT-5** 모델에 단일 API 호출로 전송하고, 10개 섹션으로 구성된 방대한 텍스트 응답을 수신합니다.

현재의 프롬프트 엔지니어링은 그 자체로 가치가 높지만, 이 모놀리식(monolithic) API 호출 방식은 시스템의 확장성과 사용자 경험에 근본적인 제약을 가합니다.

### B. 핵심 비즈니스 및 **UX** 한계 식별

현재 아키텍처의 문제는 데이터 수집의 품질이 아니라, 수집된 데이터를 처리하는 방식의 비효율성과 경직성에 있습니다. 모놀리식 API 호출 방식은 세 가지 치명적인 비즈니스 및 UX 한계를 야기합니다.

1. 견딜 수 없는 대기 시간 ("**블랙박스**" UX): 사용자가 분석을 요청한 후 결과를 받기까지 평균 15-20초의 긴 대기 시간(Long waiting time)이 발생합니다.<sup>1</sup> 이 시간 동안 사용자는 로딩 애니메이션 외에는 어떠한 실시간 피드백(No real-time feedback)도 받지 못합니다.<sup>1</sup> 이는 <sup>1</sup>에서 "**블랙박스 대기(Black-box waiting)**"로 명명된, 현대 모바일 앱에서 용납되기 어려운 치명적인 사용자 경험(UX) 저하입니다.<sup>1</sup>
2. "**All-or-Nothing**" (전부 아니면 전무) 방식의 실패: 현재 시스템은 단일 실패 지점(Single Point of Failure)을 가집니다.<sup>1</sup> 10개의 분석 섹션 중 단 하나라도 AI 모델 생성 과정에서 오류가 발생하면, API 호출 전체가 실패합니다. 이 경우 사용자는 30포인트를 소모했음에도 불구하고 아무런 결과도 얻지 못하며, 이는 직접적인 고객 불만 및 이탈로 이어집니다.
3. 경직된 UI 렌더링: 최종 결과물은 AchievementAnalysisModel의 단일 analysisContent 필드에 저장되는 거대한 텍스트 문자열(String)입니다.<sup>1</sup> 이 방식은 사용자가 요청한 7가지 핵심 시각화 요구사항(예: 최근 2개월 동향 그래프, 훈련 우선순위 체크리스트)을 근본적으로 구현 불가능하게 만듭니다. 텍스트 블록은 인터랙티브한 차트나 체크리스트로 변환될 수 없습니다.

## C. 아키텍처 전환 당위성 수립

따라서, 아키텍처 전환의 핵심 임무는 현재 데이터 흐름의 8단계, 즉 모놀리식 createChatCompletion 호출을 해체하는 것입니다.<sup>1</sup>

우리는 동기식(synchronous), 'All-or-Nothing' 방식의 단일 요청에서 비동기식(asynchronous), 장애 허용(fault-tolerant) 방식의 전문화된 인사이트 \*스트림(stream)\*으로 전환해야 합니다. 이 패러다임 전환은 <sup>1</sup>에서 처음 개념화된 3-에이전트(운동 전문가, 감성 코치, 리더) 협업 모델을 구현하기 위한 기술적 필수 요건입니다.<sup>1</sup>

## II. 'To-Be' 청사진: 3-에이전트 점진적 인사이트 시스템

## A. 하이레벨 컨셉 플로우

'To-Be' 아키텍처는 백엔드의 에이전트 분리와 프론트엔드의 점진적 UI 로딩이 완벽하게 결합된 형태입니다.

1. **Flutter UI:** 사용자가 'AI 분석' 버튼을 탭합니다.
2. **Flutter Service:** AiAnalysisScreen의 StreamBuilder가 백엔드의 SSE(Server-Sent Events) 스트림 엔드포인트를 구독하기 시작합니다.<sup>1</sup>
3. **Backend (FastAPI):** SSE 엔드포인트가 호출되면, 백엔드는 즉시 LangGraph 오케스트레이터의 astream\_events (비동기 이벤트 스트림) 메서드를 호출합니다.<sup>1</sup>
4. **LangGraph (Backend):** 오케스트레이션 그래프가 실행됩니다.
  - data\_collection\_node가 Neo4j에서 데이터를 수집합니다.
  - expert\_node (운동 전문가 에이전트)가 실행되고 완료됩니다.
5. **SSE Stream (Backend  $\rightarrow$  Frontend):** expert\_node의 완료가 감지되면, 백엔드는 첫 번째 JSON 객체(전문가 분석 결과)를 SSE 스트림을 통해 프론트엔드로 yield (방출)합니다.<sup>1</sup>
6. **Flutter UI (Stage 1):** StreamBuilder가 첫 번째 JSON 객체를 수신하고, 이를 ExpertAnalysisResult 모델로 파싱하여 UI를 즉시 재빌드합니다. 사용자는 4초 이내에 첫 번째 분석 결과를 보게 됩니다.<sup>1</sup>
7. **LangGraph (Backend):** 그래프가 계속 실행되어 coach\_node (감성 코치)가 완료됩니다.
8. **SSE Stream (Backend  $\rightarrow$  Frontend):** 백엔드가 두 번째 JSON 객체(감성 코칭 메시지)를 yield합니다.
9. **Flutter UI (Stage 2):** StreamBuilder가 두 번째 객체를 수신하고, UI 상태에 \*누적(accumulate)\*하여 감성 코칭 섹션을 추가로 렌더링합니다.<sup>1</sup>
10. **LangGraph & SSE (Stage 3):** leader\_node (리더 에이전트)가 완료되고, 최종 JSON(로드맵, 차트 데이터)이 스트림을 통해 전송되어 UI가 최종적으로 완성됩니다.

이 흐름은 <sup>1</sup>의 "블랙박스" 대기 <sup>1</sup>를 <sup>1</sup>의 "점진적 3단계 디스플레이(Progressive 3-Stage Display)"<sup>1</sup>로 완벽하게 전환시킵니다.

## B. 핵심 기술 스택 정의

이 'To-Be' 청사진을 구현하기 위한 핵심 기술 스택은 다음과 같습니다.

1. 오케스트레이션 (**LangGraph**): 에이전트 간의 복잡한 흐름, 특히 "조건부 엣지(Conditional Edges)"를 통한 강력한 오류 처리 및 'Agent-Level Fallback' 구현을 위해 선정되었습니다.<sup>1</sup>
2. 메모리 (**Neo4j**): 사용자의 목표와 루틴 간의 관계형 컨텍스트(GraphRAG) 및 러닝 기록의 시계열 패턴(TKG, Temporal Knowledge Graph)을 저장하고 검색하기 위해 선정되었습니다. 이는 단순 벡터 DB의 능력을 훨씬 뛰어넘습니다.<sup>1</sup>

3. 프론트엔드 스트리밍 (**Flutter StreamBuilder + SSE**): "블랙박스" UX를 제거하고 점진적 UI를 구현하기 위한 유일하고 가장 효율적인 프론트엔드 네이티브 솔루션입니다.<sup>1</sup>

### C. 심층 분석: 공생적 백엔드/프론트엔드 아키텍처

이 아키텍처의 핵심은 백엔드의 3-에이전트 설계<sup>1</sup>가 단순히 '업무 전문화'만을 위한 것이 아니라는 점입니다. 이는 프론트엔드의 **StreamBuilder** 아키텍처<sup>1</sup>를 가능하게 하는 직접적인 기술적 조력자입니다.

이들의 관계는 다음과 같은 논리적 흐름을 따릅니다.

1. 가장 시급한 문제는 15-20초간의 "블랙박스" UX입니다.<sup>1</sup>
2. 이에 대한 사용자 경험 목표는 "점진적 3단계 디스플레이"입니다.<sup>1</sup>
3. 이 목표를 위한 **Flutter**의 네이티브 구현체는 **StreamBuilder**입니다.<sup>1</sup>
4. **StreamBuilder**는 필연적으로 데이터의 **Stream**을 요구합니다.
5. 의미 있는 콘텐츠 스트림을 생성하는 유일한 방법은 백엔드의 모놀리식 작업을 여러 개의 논리적 '청크(chunk)'로 분할하는 것입니다.
6. <sup>1</sup>에서 제안된 3-에이전트 모델(전문가, 코치, 리더)<sup>1</sup>은 이 '청크' 분할에 가장 이상적인 논리적 단위입니다.
7. 결론적으로, 백엔드의 에이전트 분리는 프론트엔드 스트리밍 아키텍처를 구현하기 위한 필수 전제 조건입니다. 이 두 아키텍처는 서로를 가능하게 하는 공생 관계에 있습니다.

## III. 백엔드 아키텍처: 오케스트레이션 및 메모리 (LangGraph & Neo4j)

### A. 오케스트레이션 엔진: LangGraph 상태 머신

본 아키텍처는 오케스트레이션 프레임워크로 **LangGraph**를 채택합니다.<sup>1</sup>의 비교 분석에 따르면<sup>1</sup>, **CrewAI**와 같은 대안은 순차적 워크플로우 정의에는 용이하지만, 세르파 앱의 핵심 요구사항인<sup>1</sup>의 "**Agent-Level Fallback Strategy**"<sup>1</sup>를 구현하는 데 한계가 있습니다. '운동 전문가 에이전트'가 실패했을 때 동적으로 '기본 통계' 노드로 라우팅하는 복잡한 예외 처리는 **LangGraph**의 "**조건부 엣지(Conditional Edges)**" 기능을 통해서만 안정적으로 구현할 수

있습니다.<sup>1</sup>

이 시스템의 "혈류(Bloodstream)" 역할을 하는 것은 모든 에이전트(노드)가 공유하는 중앙 메모리 객체인 AgentState입니다.<sup>1</sup>

표 1: LangGraph AgentState 정의 (Python TypedDict)

- 목적: 오케스트레이션 그래프의 모든 노드가 공유할 중앙 상태 머신의 정확한 데이터 구조를 정의합니다.
- 출처: <sup>1</sup>(s. 7-8) <sup>1</sup>

| 키 (Key)         | 타입 (Python)                               | 설명                                 |
|-----------------|-------------------------------------------|------------------------------------|
| user_id         | str                                       | 컨텍스트 조회를 위한 입력 사용자 ID              |
| category        | str                                       | '운동' (향후 '학습', '대회' 등으로 확장 가능)     |
| user_data       | Optional                                  | data_collection_node가 수집한 구조화된 데이터 |
| expert_analysis | Optional[ExpertAnalysis]                  | expert_node (운동 전문가)의 JSON 출력      |
| coach_feedback  | Optional[CoachFeedback]                   | coach_node (감성 코치)의 JSON 출력        |
| final_result    | Optional                                  | leader_node (리더)의 최종 JSON 출력       |
| messages        | Annotated[List[AnyMessage], operator.add] | 디버깅 및 컨텍스트 유지를 위한 메시지 기록 누적        |
| error_message   | Optional[str]                             | 조건부 엣지가 라우팅 분기 시 사용할 오류 플래그        |
| fallback_used   | bool                                      | Fallback 로직이 실행되었음을 나타내는 플래그       |

## B. 핵심 전략: 조건부 엣지를 통한 에이전트 레벨 **Fallback**

LangGraph를 선택한 핵심 이유이자 "All-or-Nothing" 문제 <sup>1</sup>에 대한 직접적인 해결책입니다. 우리는 <sup>1</sup>에 정의된 "Fallback Strategy"를 `add_conditional_edges` <sup>1</sup>를 사용하여 구현합니다.

구현 로직은 다음과 같습니다. `expert_node` 실행 후, `route_after_expert`라는 라우팅 함수 <sup>1</sup>가 `state.get("error_message")`를 확인합니다.

- 오류 발생 시 (**True**): 그래프의 흐름을 `fallback_expert_node`로 라우팅합니다. 이 노드는 LLM을 호출하는 대신, `data_collection_node`가 수집한 통계(**statistics**) 데이터만을 기반으로 "최근 15일간 달리셨네요. 평균 페이스는 6:00/km입니다"와 같은 기본 분석을 생성합니다.
- 정상 처리 시 (**False**): 그래프의 흐름을 `coach_node`로 정상 라우팅합니다.

이 설계는 사용자가 30포인트를 지불하고도 아무것도 받지 못하는 최악의 경험을 원천적으로 차단합니다. 부분적인 성공(**Partial Success**) 및 그에 따른 부분적 포인트 차감(향후 구현)이 완전한 실패보다 비즈니스적으로 월등히 우수합니다.

## C. 장기 기억: Neo4j (GraphRAG & TKG)

AI 에이전트의 장기 기억을 위해 단순 벡터 DB가 아닌 Neo4j 지식 그래프를 도입합니다.<sup>1</sup>

1. "컨텍스트 그래프" (**GraphRAG**): <sup>1</sup>에서 지적하듯이, "10km 완주 목표"(**Goal**)와 "주 3회 조깅"(**Routine**)이 서로 연결되어 있다는 사실은 관계형 컨텍스트입니다. 이는 텍스트의 '유사성'을 찾는 벡터 검색으로는 파악할 수 없습니다. Neo4j를 사용하면 (User)-->(Goal)-->(Routine)이라는 관계 자체를 저장(GraphRAG)하여, 에이전트가 "이 사용자가 10km 완주를 위해 어떤 루틴을 설정했나요?"라는 복합적인 질문에 답할 수 있게 합니다.
2. "시계열 지식 그래프" (**TKG**): <sup>1</sup>의 `RunningRecord` <sup>1</sup>는 본질적으로 시계열 데이터입니다. <sup>1</sup>의 TKG 접근 방식 <sup>1</sup>에 따라, 우리는 이 기록들을 (User)-->(RunningRecord {date: '...', distance:...}) 형태로 그래프에 시간순으로 연결합니다. 이는 에이전트가 "사용자의 평균 페이스가 지난 2개월간 어떻게 변화했나요?"와 같은 동적 트렌드 분석을 수행할 수 있게 합니다.<sup>2</sup>

### 표 2: Neo4j Cypher 스키마 정의

- 목적: AI 에이전트의 장기 기억을 위한 지식 그래프의 구조를 정의합니다.

- 출처: <sup>1</sup> (s. 12), <sup>1</sup> (s. 12) <sup>1</sup>

| 타입        | 레이블 (Label)   | 속성 (Properties)<br>예시                   | 설명                                     |
|-----------|---------------|-----------------------------------------|----------------------------------------|
| 노드 (Node) | User          | id, name, birthYear,<br>height, weight  | GlobalUser 모델 <sup>1</sup>             |
| 노드 (Node) | Goal          | id, name, category,<br>targetValue      | GoalModel 모델 <sup>1</sup>              |
| 노드 (Node) | Routine       | id, name, category,<br>frequency        | RoutineModel 모델<br><sup>1</sup>        |
| 노드 (Node) | RunningRecord | id, date,<br>distanceKm,<br>averagePace | RunningRecord<br>모델 (TKG) <sup>1</sup> |
| 엣지 (Edge) | HAS_GOAL      |                                         | (User)-->(Goal)                        |
| 엣지 (Edge) | LINKED_TO     |                                         | (Goal)-->(Routine)                     |
| 엣지 (Edge) | LOGGED_RUN    |                                         | (User)-->(RunningR<br>ecord)           |

## IV. 백엔드 구현: 에이전트, 도구 및 프롬프트

### A. 데이터 수집: **AIAnalysisDataCollector** 리팩토링

현재 아키텍처 <sup>1</sup>의 **AIAnalysisDataCollector** 로직은 매우 가치가 있지만, 모놀리식 함수 내에 갇혀 있습니다. <sup>11</sup>의 핵심 제안에 따라, 이 로직을 "시계열 RAG" 전처리 단계로 재정의하고, **LangGraph**의 **data\_collection\_node**가 호출할 수 있는 개별 **@tool** 세트로 리팩토링합니다.<sup>1</sup>

이는 데이터 수집(도구의 역할)과 데이터 분석(LLM 에이전트의 역할)을 명확히 분리하는(Separation of Concerns) 핵심적인 아키텍처 개선입니다.

표 3: **AIAnalysisDataCollector** 도구 명세서 (**Python @tool**)

- 목적: `data_collection_node`가 호출할 수 있는 Python 함수의 명확한 인터페이스를 정의합니다.
- 출처: <sup>1</sup>(s. 3-4) <sup>1</sup> 및 <sup>1</sup>(s. 13)의 로직을 리팩토링함

| 도구 (@tool)                           | 설명                                                                          | 입력 (Input)                  | 출력 (Output)                                 |
|--------------------------------------|-----------------------------------------------------------------------------|-----------------------------|---------------------------------------------|
| <code>get_user_context</code>        | Neo4j에서 사용자의 User, Goal, Routine 노드 및 관계를 조회 (GraphRAG) <sup>1</sup>        | <code>user_id: str</code>   | dict (userInfo, goals, routines)            |
| <code>get_running_statistics</code>  | Neo4j TKG에서 2개월간의 통계(총 거리, 평균 페이스 등)를 계산 <sup>1</sup>                       | <code>user_id: str</code>   | dict (statistics)                           |
| <code>get_recent_running_logs</code> | Neo4j TKG에서 최신 10개의 RunningRecord 노드를 조회 <sup>1</sup>                       | <code>user_id: str</code>   | List                                        |
| <code>get_recent_trend</code>        | 최근 7개 vs. 이전 7개 기록의 평균 거리를 비교 <sup>1</sup>                                  | <code>user_id: str</code>   | str ("증가", "유지", "감소")                      |
| <code>validate_data</code>           | 수집된 데이터가 <sup>1</sup> (s. 4)의 유효성 기준(예: 러닝 기록 3개 이상)을 충족하는지 검증 <sup>1</sup> | <code>data: UserData</code> | (bool, List[str]) (isValid, failureReasons) |

**B. 에이전트 전문화: SOTA LLM 라우팅 (2025년 11월 기준)**

단일 고비용 "GPT-5" 모델<sup>1</sup>을 모든 작업에 사용하는 비효율적인 방식에서 벗어나, <sup>11</sup>에서 제안된 정교한 모델 라우팅 전략을 채택합니다. 이는 작업의 복잡성에 맞는 최적의 LLM을 사용하여 비용과 성능을 모두 최적화합니다.

표 4: SOTA LLM 라우팅 매트릭스 (2025년 11월 기준)

- 목적: 각 에이전트 노드에 비용 및 성능이 최적화된 LLM을 할당합니다.
- 출처: <sup>1</sup>(s. 5, 표 2) <sup>1</sup>

| 에이전트 노드         | 주요 임무                                     | 권장 모델 (2025)                    | 근거                                                   |
|-----------------|-------------------------------------------|---------------------------------|------------------------------------------------------|
| 1. Expert Agent | 심층 데이터 분석, 통계 해석, 추론, 4주 로드맵 초안 생성        | Claude 4 Opus 또는 Gemini 2.5 Pro | 업계 최고 수준의 추론(Reasoning) 및 분석 능력 필요 <sup>1</sup>      |
| 2. Coach Agent  | 공감, 창의적 글쓰기, "세르파" 톤 적용, 부정적 피드백의 긍정적 재구성 | GPT-4o 또는 Claude 3.5 Sonnet     | 높은 감성 지능(EQ), 창의성, 빠른 응답 속도 필요 <sup>1</sup>          |
| 3. Leader Agent | 입력(전문가, 코치) 종합, 최종 JSON 스키마 포매팅, 요약       | Claude 3.5 Sonnet (권장)          | 뛰어난 요약/종합 능력, 2배 빠른 속도, 높은 신뢰성 및 비용 효율성 <sup>1</sup> |

C. 에이전트 노드 구현 & 프롬프트 엔지니어링

기존의 10섹션 모놀리식 프롬프트<sup>1</sup>는 3개의 전문화된 에이전트 프롬프트로 대체됩니다. <sup>11</sup>의 제안에 따라, LLM의 출력을 안정적인 JSON 객체로 강제하기 위해 XML 태그와 명시적인 JSON 스키마를 사용합니다.

1. expert\_node 시스템 프롬프트 (예시):<sup>1</sup>

XML

당신은 15년 경력의 프로 러닝 코치이며, 스포츠 과학 석사 학위를 보유한 데이터 분석 전문가입니다.

다음은 <user\_data> 태그에 제공된 러너의 상세 데이터입니다.

<user\_data>

{state.user\_data}

</user\_data>

당신의 임무는 이 데이터를 심층 분석하여 다음 4가지 섹션으로 구성된 전문적인 분석 리포트를 생성하는 것입니다.

분석은 반드시 <user\_data>의 수치(통계, 목표, 신체 정보)에 기반해야 합니다.

반드시 다음 JSON 스키마를 준수하여 응답해야 합니다. 다른 텍스트는 절대 추가하지 마십시오.

```
{
  "patternEvaluation": "...",
  "strengths": ["...", "..."],
  "improvements": ["...", "..."],
  "trendAnalysis": "..."
}
```

## 2. coach\_node 시스템 프롬프트 (예시):<sup>1</sup>

XML

당신은 세르파의 따뜻하고 공감 능력이 뛰어난 러닝 코치입니다. 항상 부드럽고 친근한 존댓말을 사용합니다.

전문가 분석 결과가 <expert\_analysis> 태그로 주어집니다.

<expert\_analysis>

{state.expert\_analysis}

</expert\_analysis>

사용자는 '도전적 성향'입니다.

전문가 분석의 <improvements> 섹션(개선점)을 사용자가 부담스럽지 않고 긍정적으로 받아들일 수 있도록,

"이것만 보완하면 훨씬 더 성장할 수 있다"는 톤을 유지하며 따뜻한 격려와 동기부여 메시지로 바꾸어 주세요.

다음 JSON 형식을 준수하여 응답하세요.

```
{
  "encouragement": "강점에 기반한 따뜻한 응원 메시지",
  "empathyMessage": "개선점에 대한 공감적 피드백"
}
```

### 3. leader\_node 시스템 프롬프트 (예시):<sup>1</sup>

XML

당신은 전문가 분석과 감성 피드백을 종합하는 리더 에이전트입니다.

<user\_data>, <expert\_analysis>, <coach\_feedback> 태그의 모든 정보를 종합하세요.

<user\_data>

{state.user\_data}

</user\_data>

<expert\_analysis>

{state.expert\_analysis}

</expert\_analysis>

<coach\_feedback>

{state.coach\_feedback}

</coach\_feedback>

이제, <user\_data>의 목표와 <expert\_analysis>의 분석 결과를 바탕으로 다음 2가지 최종 결과물을 생성하세요:

1. 4주간의 단계별 목표 달성 로드맵

2. 다음 주 실천 계획 (Top 5 우선순위)

(만약 state['fallback\_used']가 True라면, 'weeklyRoadmap'과 'actionPlan' 대신 기본 권장사항(예: '주 3회 러닝 유지')을 생성해야 함)

다음 JSON 형식을 준수하여 응답하세요.

```
{
  "gapAnalysis": "현재 <statistics>와 목표('goals') 간의 Gap을 구체적으로 분석",
  "weeklyRoadmap": [
    {"week": 1, "summary": "...", "details": "..."},
    {"week": 2, "summary": "...", "details": "..."}
  ],
  "actionPlan": [
    {"id": "...", "priority": 1, "task": "..."},
    {"id": "...", "priority": 2, "task": "..."}
  ],
  "trendVisualization": {
    "type": "line_chart",
    "data": [{"x": 0, "y": 6.5}, {"x": 1, "y": 6.3}]
  }
}
```

## V. 프론트엔드 아키텍처: 점진적 UX 스트림 (Flutter)

### A. 핵심 패턴: **StreamBuilder Over FutureBuilder**

프론트엔드 아키텍처의 가장 중대한 변화는 FutureBuilder에서 StreamBuilder로의 전환입니다.<sup>1</sup> 이는 "블랙박스" UX<sup>1</sup>를 해결하기 위한 유일한 네이티브 솔루션입니다.

- **FutureBuilder (As-Is):** 단일 Future<AchievementAnalysisModel>가 완료될 때까지 (15-20초) 무한 대기합니다.
- **StreamBuilder (To-Be):** Stream<IAgentResult>를 구독하고, 스트림을 통해 새로운 데이터 조각이 *yield*될 때마다 (예: 4초, 7초, 10초 시점) UI를 즉시 재빌드합니다.<sup>1</sup>

### B. 데이터 전송: 3단계 디스플레이를 위한 **SSE (Server-Sent Events)**

StreamBuilder에 데이터를 공급하기 위해 SSE 기술을 사용합니다.<sup>1</sup>

- **백엔드 (FastAPI/Python):** <sup>11</sup>에서 제안된 스트리밍 엔드포인트를 구현합니다. 이 엔드포인트는 LangGraph의 `app.ostream_events()`를 구독하고, "expert", "coach", "leader" 노드의 `on_chain_end` 이벤트가 발생할 때마다 해당 노드의 출력 JSON을 클라이언트로 *yield* (전송)합니다.<sup>5</sup>
- **프론트엔드 (Flutter/Dart):** <sup>11</sup>에서 제안된 MultiAgentOrchestrator 서비스를 구현합니다. 이 서비스는 `http` 패키지의 `http.Client`를 사용하여 SSE 엔드포인트에 연결하고, `response.stream.transform(utf8.decoder)`를 통해 수신되는 데이터 chunk를 리스닝합니다. 각 chunk (JSON 문자열)를 파싱하여 Dart 객체로 변환한 후, Stream<IAgentResult>를 통해 *yield*합니다.

### C. 데이터 모델링 (Dart): IAgentResult 인터페이스 및 Freezed 클래스

백엔드에서 스트리밍되는 3가지 다른 유형의 JSON 객체를 하나의 Stream으로 처리하기 위해,

<sup>11</sup>에 정의된 타입-안전(**type-safe**) Dart 모델을 사용합니다.

1. 공통 인터페이스 (**Interface**): 모든 에이전트 결과가 구현할 추상 클래스를 정의합니다. Fallback 시나리오 <sup>1</sup>를 감지하기 위해 **fallback** 플래그를 포함합니다.

Dart

```
// lib/features/analysis/models/agent_response.model.dart
abstract class IAgentResult {
  bool get fallback;
}
```

2. **freezed** 모델: **freezed**와 **json\_serializable**을 사용하여 각 에이전트의 출력 JSON 스키마에 정확히 매핑되는 Dart 클래스를 생성합니다.<sup>1</sup>
  - @freezed class ExpertAnalysisResult with... implements IAgentResult: patternEvaluation, strengths, improvements, statistics 필드 포함.<sup>1</sup>
  - @freezed class EmotionalCoachResult with... implements IAgentResult: encouragement, empathyMessage 필드 포함.<sup>1</sup>
  - @freezed class LeaderAgentResult with... implements IAgentResult: gapAnalysis, trendVisualization 필드 포함.<sup>1</sup>
3. 중첩 **JSON** 처리: LeaderAgentResult는 List<WeekPlan>과 List<ActionItem>이라는 중첩된 JSON 배열을 포함합니다.<sup>1</sup> json\_serializable이 이를 올바르게 파싱하도록 부모 클래스에 @JsonSerializable(explicitToJson: true) 어노테이션을 추가해야 합니다.<sup>1</sup>

## VI. 프론트엔드 구현: 7가지 분석 시각화

이 섹션은 LeaderAgentResult 및 EmotionalCoachResult 객체의 구조화된 데이터를 사용자가 요청한 7가지 시각화 요구사항으로 변환하는 구체적인 Flutter 위젯 구현 방안을 제시합니다.

### A. 트렌드 및 로그 (시각화 1, 2, 3): fl\_chart

사용자의 최근 러닝 기록과 2개월간의 동향(거리, 페이스 변화)은

LeaderAgentResult.trendVisualization 맵 <sup>1</sup>에 담긴 데이터를 사용하여 시각화합니다.

<sup>11</sup>에서 제안된 TrendLineChart 위젯을 구현합니다. 이 위젯은 2025년 Flutter 생태계 표준인 fl\_chart 패키지를 활용합니다.<sup>1</sup> 핵심 변환 로직은 LeaderAgentResult.trendVisualization['data'] (예: [{"x": 0, "y": 6.5}, {"x": 1, "y": 6.3}]) 리스트를 순회하며, 각 JSON 맵을 fl\_chart가 요구하는

FIspot(x, y) 객체 리스트로 변환하는 것입니다.<sup>1</sup>

Dart

```
// lib/features/analysis/widgets/trend_chart.widget.dart
//
List<FIspot> _getSpots() {
  final List<dynamic> dataPoints = chartData['data']??;
  List<FIspot> spots =;
  for (var i = 0; i < dataPoints.length; i++) {
    final point = dataPoints[i] as Map<String, dynamic>;
    spots.add(FIspot(
      (point['x'] as num).toDouble(),
      (point['y'] as num).toDouble(),
    ));
  }
  return spots;
}
//...
LineChart(
  LineChartData(
    lineBarsData:,
  ),
)
```

## B. 로드맵 및 실행 계획 (시각화 4, 5): ListView.builder 및 CheckboxListTile

4주 훈련 계획과 우선순위별 실천 계획은 LeaderAgentResult의 중첩 리스트 데이터를 활용하여 구현합니다.

1. 훈련 로드맵 (시각화 4):<sup>11</sup>의 WeeklyRoadmapList 위젯을 구현합니다. 이 위젯은 LeaderAgentResult.weeklyRoadmap (List<WeekPlan>)을 입력받아 ListView.builder를 사용해 렌더링합니다. 각 WeekPlan 객체는 Card 위젯 내의 ListTile로 변환되어, plan.summary (예: "주 3회, 5km")와 plan.details (예: "페이스 6:30/km 유지")를 명확하게 표시합니다.<sup>1</sup>

2. 실천 계획 (시각화 5): <sup>11</sup>의 `ActionChecklist` 위젯을 구현합니다. 이는 단순한 텍스트 표시가 아닌, 사용자의 \*상호작용(체크)\*을 처리해야 하므로 반드시 `StatefulWidget`으로 구현되어야 합니다.<sup>1</sup>
  - `Map<String, bool> _checkedState = {};` 상태 변수를 사용하여 각 항목의 체크 여부를 관리합니다.
  - `LeaderAgentResult.actionPlan (List<ActionItem>)` 리스트를 `ListView.builder`로 순회하며, 각 `ActionItem`을 `CheckboxListTile` 위젯으로 렌더링합니다.<sup>1</sup>
  - `CheckboxListTile`의 `value`는 `_checkedState[item.id]?? false`에 바인딩되고, `onChanged` 콜백은 `setState`를 호출하여 `_checkedState` 맵을 업데이트합니다.<sup>1</sup>

## C. 신체 관리 및 감성 코칭 (시각화 6, 7)

- 신체 관리/회복 (시각화 6): 이는 `LeaderAgentResult.actionPlan` 내의 우선순위가 높은 항목(예: `priority: 2, task: "러닝 후 10분 폼롤러 스트레칭"`)으로 포함되거나, `gapAnalysis` 텍스트의 일부로 제공되어 `ActionChecklist` 또는 별도의 텍스트 위젯을 통해 렌더링됩니다.
- 감성 코치 피드백 (시각화 7): 이는 `EmotionalCoachResult` 객체의 `encouragement` 및 `empathyMessage` 필드 <sup>1</sup>에서 직접 가져온 텍스트를 `_buildCoachUI` <sup>1</sup>와 같은 전용 위젯을 통해 사용자에게 따뜻한 '세르파 톤'으로 표시합니다.

## D. 핵심 프론트엔드 로직: 상태 누적 (State Accumulation)

`StreamBuilder` 사용 시 발생하는 마지막 과제는 "어떻게 UI를 대체하지 않고 누적할 것인가?"입니다. `coachResult`가 도착했을 때 `expertResult` UI가 사라지면 안 됩니다.

이에 대한 해결책은 <sup>11</sup>에 제시된 "상태 누적(State Accumulation)" 패턴입니다.<sup>1</sup>

1. `AIAnalysisScreen`을 `ConsumerWidget`이 아닌 `ConsumerStatefulWidget`으로 변경합니다.<sup>1</sup>
2. `State` 클래스 내에 누적용 리스트를 선언합니다: `final List<IAgentResult> _results = ;`<sup>1</sup>
3. `StreamBuilder`의 `builder` 함수 내부에서, 새 데이터(`snapshot.data`)가 도착할 때마다 `_results` 리스트에 추가합니다. (중복 방지 로직 포함)

```
Dart
//
if (snapshot.hasData) {
  if (!_results.any((res) => res.runtimeType == snapshot.data.runtimeType)) {
    _results.add(snapshot.data!);
  }
}
```

4. UI를 빌드하는 `_buildProgressiveDisplay` 함수<sup>1</sup>는 `snapshot.data`를 직접 사용하는 대신, `_results` 리스트 전체를 기반으로 렌더링합니다.

Dart

//

```
Widget _buildProgressiveDisplay(List<IAgentResult> results,...) {  
  final expertResult = results.whereType<ExpertAnalysisResult>().firstOrNull;  
  final coachResult = results.whereType<EmotionalCoachResult>().firstOrNull;  
  final leaderResult = results.whereType<LeaderAgentResult>().firstOrNull;  
  
  return ListView(  
    children: ,  
  );  
}
```

이 패턴을 통해 "점진적 3단계 디스플레이"<sup>1</sup> 요구사항이 코드 레벨에서 완벽하게 충족됩니다.

## VII. 엔드투엔드 구현 및 전환 로드맵

### A. 통합 플로우 (요약)

본 청사진은 현재의 15-20초 "블랙박스" 시스템을 7-10초<sup>1</sup> 내에 3단계로 결과를 보여주는 "점진적 투명 박스" 시스템으로 전환합니다.

전체 흐름은 Flutter UI  $\rightarrow$  StreamBuilder  $\rightarrow$  SSE 요청  $\rightarrow$  FastAPI 엔드포인트  $\rightarrow$  LangGraph app.astream\_events  $\rightarrow$  data\_collection\_node (Neo4j TKG/GraphRAG 도구 호출)  $\rightarrow$  expert\_node (Claude 4 호출)  $\rightarrow$  SSE-yield (전문가)  $\rightarrow$  coach\_node (GPT-4o 호출)  $\rightarrow$  SSE-yield (코치)  $\rightarrow$  leader\_node (Claude 3.5 호출)  $\rightarrow$  SSE-yield (리더)  $\rightarrow$  StreamBuilder 수신, IAgentResult 파싱, \_results 리스트 누적  $\rightarrow$  Flutter UI 3단계 재빌드의 순서로 실행됩니다.

### B. 업데이트된 전환 로드맵 (6주)

<sup>11</sup>의 초기 로드맵과 <sup>11</sup>의 성공 기준(지연 시간 10초 미만, 오류율 5% 미만)을 기반으로, 본 청사진을 실행하기 위한 6주간의 상세 전환 로드맵을 다음과 같이 제안합니다.

- **1주차: 백엔드 기반 및 데이터 노드:**
  - Neo4j 스키마(표 2)를 프로덕션 환경에 배포합니다.
  - LangGraph AgentState(표 1)를 정의하고 그래프 빌드를 시작합니다.
  - AIAnalysisDataCollector <sup>1</sup>의 로직을 리팩토링하여 data\_collection\_node와 @tool 세트(표 3)를 구현합니다.
- **2주차: 핵심 에이전트 및 Fallback 구현:**
  - expert\_node (Claude 4)와 leader\_node (Claude 3.5)를 구현합니다 (표 4).
  - (핵심) fallback\_expert\_node 및 route\_after\_expert 조건부 엣지(III.B)를 구현하여 "All-or-Nothing" 문제를 우선적으로 해결합니다.
- **3주차: 백엔드-프론트엔드 브릿지 (SSE):**
  - FastAPI SSE 스트리밍 엔드포인트<sup>1</sup>를 구현합니다.
  - Flutter MultiAgentOrchestrator 서비스<sup>1</sup>를 구현하여 SSE 스트림 수신을 테스트합니다.
  - IAgentResult 및 freezed 모델(V.C)을 Dart 코드베이스에 구현합니다.
- **4주차: 프론트엔드 UI 구현:**
  - AiAnalysisScreen을 FutureBuilder에서 StreamBuilder로 전환합니다.
  - 상태 누적 로직<sup>1</sup>을 구현합니다.
  - 7가지 시각화 요구사항을 위한 위젯(TrendLineChart <sup>1</sup>, WeeklyRoadmapList <sup>1</sup>, ActionChecklist <sup>1</sup>)을 구현합니다.
- **5주차: 감성 코치 에이전트 및 통합 테스트:**
  - coach\_node (GPT-4o)를 구현하여 '세르파 톤'을 적용합니다.
  - 엔드투엔드(E2E) 테스트를 수행합니다. 특히, expert\_node를 강제로 실패시켜 fallback\_expert\_node가 정상적으로 트리거되고 UI에 기본 분석이 표시되는지 <sup>1</sup> 집중적으로 테스트합니다.
- **6주차: 성능 튜닝 및 배포:**
  - LangSmith <sup>1</sup> 또는 유사한 도구를 사용하여 각 노드의 지연 시간, 토큰 사용량, 비용을 모니터링합니다.
  - 프롬프트를 미세 조정하여 전체 지연 시간을 10초 미만으로 최적화합니다.
  - 기존 모놀리식 아키텍처와 A/B 테스트 <sup>1</sup>를 시작하여 사용자 만족도 및 오류율을 비교 검증한 후, 점진적으로 100% 롤아웃합니다.

## 참고 자료

1. AI\_ANALYSIS\_SYSTEM\_ARCHITECTURE.pdf
2. This Week in Neo4j: MCP, GraphRAG, Knowledge Graph, Temporal Graph and more, 11월 13, 2025에 액세스,  
<https://neo4j.com/blog/twin4j/this-week-in-neo4j-mcp-graphrag-knowledgegraph-temporal-graph-and-more/>
3. Graphiti: Knowledge Graph Memory for an Agentic World - Neo4j, 11월 13,

2025에 액세스,

<https://neo4j.com/blog/developer/graphiti-knowledge-graph-memory/>

4. Video: 102 Temporal Graph Analysis - NODES2022 - Fabio Montagna - Neo4j, 11월 13, 2025에 액세스,  
<https://neo4j.com/videos/102-temporal-graph-analysis-nodes2022-fabio-montagna/>
5. Streaming from langgraph-supervisor - LangChain Forum, 11월 13, 2025에 액세스,  
<https://forum.langchain.com/t/streaming-from-langgraph-supervisor/1789>
6. LangGraph HITL – Upgrading FastAPI Feedback Loops with Token Streaming and SSE, 11월 13, 2025에 액세스, <https://www.youtube.com/watch?v=xRnDptSE6xE>
7. Streaming with events stream API using FastAPI #17240 - GitHub, 11월 13, 2025에 액세스, <https://github.com/langchain-ai/langchain/discussions/17240>
8. Langgraph: How to stream updates from an already running graph? : r/LangChain - Reddit, 11월 13, 2025에 액세스,  
[https://www.reddit.com/r/LangChain/comments/1k4bkx7/langgraph\\_how\\_to\\_stream\\_updates\\_from\\_an\\_already/](https://www.reddit.com/r/LangChain/comments/1k4bkx7/langgraph_how_to_stream_updates_from_an_already/)